



Module : Systèmes répartis et programmation parallèle

Rapport de projet

Mini moteur MapReduce avec MPI

Implémentation de Map, Shuffle, Reduce et comparaison avec Apache Spark

Réalisé par	ERFOUDI Anass JOUNAIDI Oussama MOUSDIK Ismail
Université	Université Hassan II
Module	Systèmes répartis et programmation parallèle
Année universitaire	2025-2026

Résumé

Ce rapport présente la conception, l'implémentation et l'évaluation d'un mini moteur MapReduce développé avec MPI en Python. Le projet traite le problème classique du Word Count afin d'illustrer les étapes fondamentales du traitement distribué : la division des données, le calcul local, la redistribution des clés et l'agrégation finale. Une version équivalente avec Apache Spark est également développée pour comparer une approche bas niveau, centrée sur la communication explicite entre processus, avec une approche haut niveau orientée Big Data.

Le moteur MPI utilise mpi4py, Open MPI et un schéma de redistribution déterministe basé sur un hachage stable du mot. Les résultats obtenus montrent que les sorties MPI et Spark sont identiques sur le fichier de test `sample_large.txt`, ce qui valide la correction fonctionnelle. Le benchmark local indique que MPI termine le traitement en environ 0,47 à 0,51 seconde selon le nombre de processus, tandis que Spark local[*] prend environ 10,57 secondes dans l'environnement testé. Cette différence s'explique principalement par le coût de démarrage de Spark et la petite taille du fichier de test. Le projet inclut aussi une interface web Streamlit permettant de lancer les traitements, visualiser les résultats et présenter le benchmark de manière interactive.

Mots-clés : MapReduce, MPI, mpi4py, Apache Spark, PySpark, Streamlit, Word Count, traitement distribué, programmation parallèle.

Abstract

This report presents the design, implementation and evaluation of a mini MapReduce engine developed with MPI in Python. The project focuses on the Word Count problem to demonstrate the main stages of distributed processing: data partitioning, local computation, key redistribution and final aggregation. An equivalent Apache Spark implementation is also provided to compare a low-level communication-oriented approach with a high-level Big Data framework.

The MPI engine uses mpi4py, Open MPI and a deterministic shuffle strategy based on a stable word hash. Experimental results show that MPI and Spark produce identical outputs on `sample_large.txt`, validating functional correctness. The local benchmark shows MPI execution times around 0.47-0.51 seconds, while Spark local[*] takes about 10.57 seconds in the tested environment. This difference is mainly due to Spark startup overhead and the limited input size. A Streamlit web interface is included to run the processing, visualize word-count results and present the benchmark interactively.

Table des matières

- 1. Introduction** - Contexte, motivation et objectifs
- 2. Cadre théorique** - MapReduce, MPI et Apache Spark
- 3. Analyse et conception** - Architecture logicielle et organisation du projet
- 4. Implémentation du moteur MPI** - Lecture, scatter, map, shuffle, reduce, gather
- 5. Implémentation Spark** - Pipeline PySpark et réduction par clé
- 6. Interface web Streamlit** - Démonstration, upload, exécution et visualisation
- 7. Protocole expérimental** - Environnement, commandes et métriques
- 8. Résultats et discussion** - Benchmark, exactitude et interprétation
- 9. Limites et perspectives** - Améliorations possibles
- 10. Conclusion** - Synthèse des acquis du projet
- Annexes** - Commandes, sorties et extraits de code

1. Introduction

Le traitement de données volumineuses occupe une place centrale dans les systèmes distribués modernes. Les applications web, les plateformes scientifiques, les journaux d'événements et les systèmes d'information produisent des fichiers dont le traitement séquentiel devient rapidement insuffisant. Pour répondre à cette limite, les approches parallèles et distribuées permettent de diviser le travail entre plusieurs unités de calcul, de traiter chaque portion en parallèle, puis de fusionner les résultats.

Le modèle MapReduce est un paradigme simple et puissant pour structurer ce type de calcul. Il sépare le traitement en trois phases logiques : Map, Shuffle et Reduce. Dans le cadre de ce projet, le modèle est implémenté manuellement avec MPI afin de rendre visibles les mécanismes internes de communication entre processus. La même opération est ensuite réalisée avec Apache Spark, un framework Big Data de plus haut niveau, pour comparer les deux approches.

1.1 Problématique

La problématique traitée est la suivante : comment distribuer le comptage des occurrences de mots dans un fichier texte entre plusieurs processus, tout en garantissant que chaque mot est compté correctement et que le résultat final reste identique à celui produit par une solution Big Data de référence comme Spark ?

1.2 Objectifs du projet

- implémenter un mini moteur MapReduce avec MPI et mpi4py ;
- matérialiser les phases Map, Shuffle et Reduce ;
- résoudre le problème Word Count sur un fichier texte ;
- développer une version de comparaison avec Apache Spark ;
- mesurer les temps d'exécution pour plusieurs configurations ;
- créer une interface web Streamlit pour la démonstration ;
- produire des résultats exploitables dans un rapport et une présentation.

2. Cadre théorique

2.1 Le modèle MapReduce

MapReduce est un modèle de programmation conçu pour exprimer des traitements distribués sous une forme simple. L'idée principale consiste à transformer un grand ensemble de données en paires clé-valeur, regrouper les valeurs associées à la même clé, puis appliquer une opération d'agrégation. Le Word Count illustre parfaitement ce principe : chaque mot devient une clé et chaque occurrence vaut 1.

Pipeline MapReduce implémenté avec MPI

Le projet matérialise explicitement les communications : scatter, alltoall et gather.

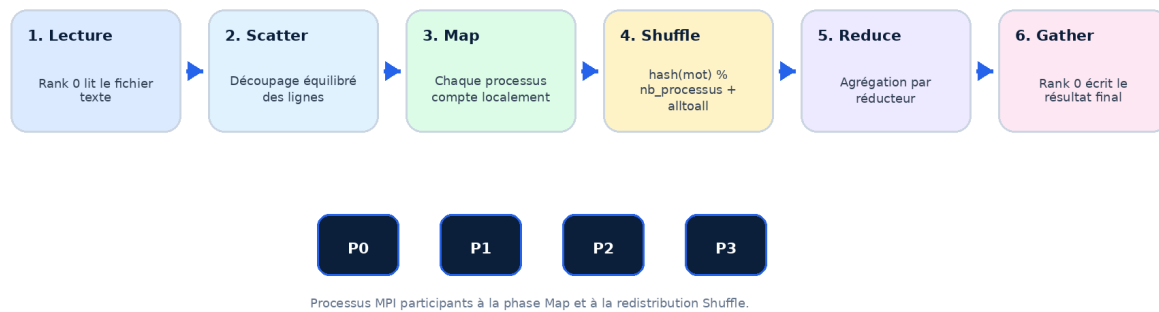


Figure 1 - Pipeline MapReduce mis en œuvre dans le moteur MPI.

2.2 Phase Map

La phase Map est exécutée localement par chaque processus. Dans ce projet, chaque processus reçoit un sous-ensemble de lignes du fichier, nettoie les mots, puis calcule un dictionnaire local de la forme mot -> nombre d'occurrences. Cette étape ne nécessite pas encore de communication entre processus, car chaque processus travaille uniquement sur sa portion de données.

2.3 Phase Shuffle

La phase Shuffle est la plus importante du point de vue distribué. Les mots identiques peuvent apparaître dans plusieurs partitions du fichier ; ils doivent donc être envoyés vers le même processus réducteur. Le projet utilise la formule $\text{stable_word_hash}(\text{word}) \% \text{number_of_processes}$ afin de déterminer le processus responsable de chaque mot. Cette redistribution est réalisée par une communication collective alltoall.

2.4 Phase Reduce

Après le Shuffle, chaque processus reçoit un ensemble de mots dont il devient responsable. Il additionne les compteurs reçus pour produire une partie du résultat final. Le processus de rang 0 rassemble ensuite les dictionnaires réduits et écrit le fichier de sortie.

2.5 MPI et Apache Spark

MPI est une interface de passage de messages qui donne au programmeur un contrôle explicite sur la communication entre processus. Cette approche est adaptée à l'apprentissage du parallélisme car chaque échange de données est visible dans le code. Apache Spark, au contraire, propose une abstraction de haut niveau : le développeur décrit les transformations à appliquer aux données, tandis que Spark gère la planification, les partitions et les communications internes.

3. Analyse et conception du projet

3.1 Architecture générale

Le projet est organisé autour de deux chaînes de traitement : une chaîne MPI et une chaîne Spark. Les deux lisent un fichier texte, exécutent un Word Count, écrivent un fichier résultat et alimentent ensuite les visualisations de l'interface Streamlit.

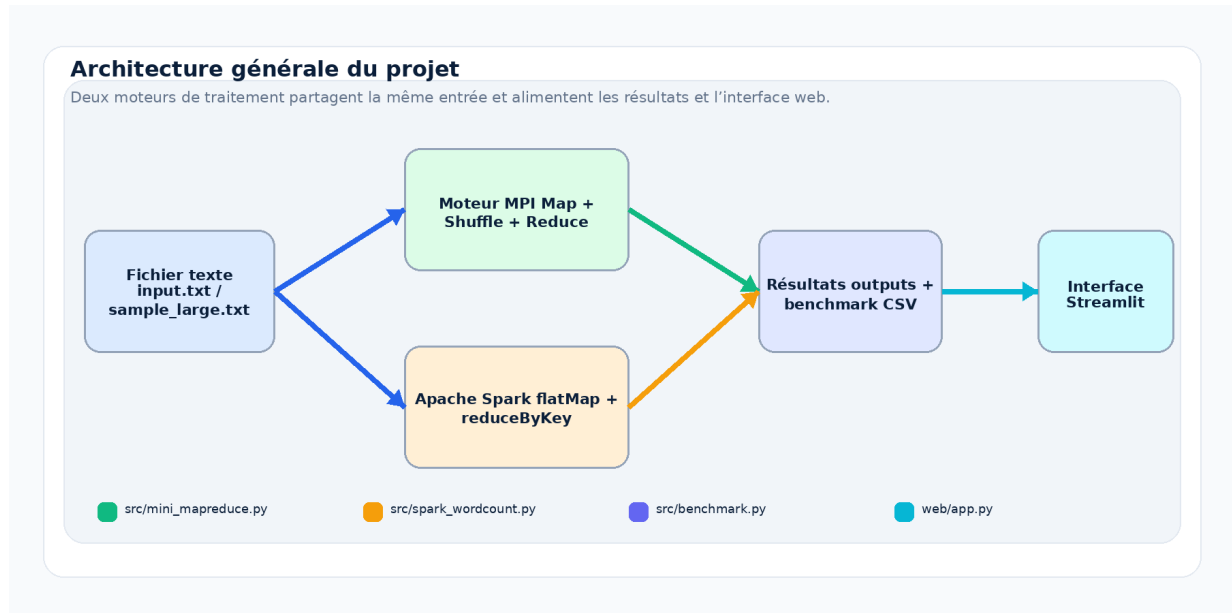


Figure 2 - Architecture générale du projet mini-mapreduce-mpi-web.

3.2 Organisation des dossiers

```

mini-mapreduce-mpi-web/
├── data/
│   ├── input.txt
│   └── sample_large.txt
├── src/
│   ├── mini_mapreduce.py
│   ├── spark_wordcount.py
│   ├── benchmark.py
│   └── utils.py
├── web/
│   └── app.py
├── results/
│   ├── mpi_output.txt
│   ├── spark_output.txt
│   └── comparison.csv
├── requirements.txt
└── README.md
  
```

Fichier	Rôle dans le projet
mini_mapreduce.py	Implémentation MPI du moteur MapReduce : lecture, scatter, map, shuffle, reduce et gather.
spark_wordcount.py	Version Apache Spark du Word Count avec flatMap, map et reduceByKey.
benchmark.py	Script de comparaison automatique entre MPI 2/4/6 processus et Spark local[*].
utils.py	Fonctions partagées : nettoyage des mots, tokenisation, lecture et écriture des résultats.
web/app.py	Interface Streamlit : upload, lancement des commandes, tableaux, graphiques et explications.
results/comparison.csv	Fichier CSV contenant les temps d'exécution expérimentaux.

3.3 Choix technologiques

Technologie	Justification
Python 3	Langage principal, simple à expliquer et adapté aux prototypes pédagogiques.
mpi4py / Open MPI	Communication entre processus MPI depuis Python.
PySpark / Apache Spark	Comparaison avec un framework Big Data haut niveau.
Streamlit	Interface web rapide pour démonstration interactive.
pandas / matplotlib	Lecture des résultats et génération de graphiques.

4. Implémentation du moteur MapReduce avec MPI

4.1 Lecture et distribution des données

Le processus de rang 0 lit le fichier d'entrée, le découpe en lignes, puis divise ces lignes en plusieurs chunks équilibrés. La fonction `chunk_lines` distribue les lignes de façon cyclique afin d'éviter qu'un seul processus reçoive toute la charge. Les morceaux sont ensuite envoyés aux processus par `comm.scatter`.

```
if rank == 0:
    text = read_text_file(input_file)
    lines = text.splitlines()
    chunks = chunk_lines(lines, size)
else:
    chunks = None

local_lines = comm.scatter(chunks, root=0)
```

4.2 Phase Map : comptage local

Chaque processus applique la fonction `local_map` sur les lignes qu'il a reçues. Les mots sont nettoyés en minuscules et débarrassés de la ponctuation. Le résultat local est un dictionnaire de fréquences.

```
def local_map(lines):
    counter = Counter()
    for line in lines:
        counter.update(tokenize_text(line))
    return dict(counter)
```

4.3 Phase Shuffle : redistribution déterministe

Le Shuffle garantit que toutes les occurrences d'un même mot sont envoyées au même processus réducteur. Pour éviter le comportement non déterministe du hash Python standard entre exécutions, le projet utilise MD5 comme hachage stable.

```
def stable_word_hash(word):
    digest = hashlib.md5(word.encode("utf-8")).hexdigest()
    return int(digest, 16)

destination_process = stable_word_hash(word) % size
```

Chaque processus construit une liste de buckets, un bucket par destination. Ensuite, `comm.alltoall` échange ces buckets entre tous les processus. Cette instruction est au cœur du projet car elle matérialise la phase Shuffle de MapReduce.

4.4 Phase Reduce et collecte finale

Après la redistribution, chaque processus agrège les compteurs qu'il a reçus. Les dictionnaires partiels sont ensuite rassemblés par `comm.gather` au rang 0. Celui-ci fusionne les résultats et écrit un fichier trié par fréquence décroissante.

```
incoming_buckets = comm.alltoall(outgoing_buckets)
reduced_counts = reduce_counts(incoming_buckets)
gathered_counts = comm.gather(reduced_counts, root=0)
```

5. Implémentation de la version Apache Spark

La version Spark réalise la même opération Word Count, mais avec une API de haut niveau. Spark lit le fichier sous forme de RDD, découpe les lignes en mots, nettoie les tokens, transforme chaque mot en paire (mot, 1), puis utilise `reduceByKey` pour additionner les valeurs par clé.

```
counts = (
    lines.flatMap(lambda line: line.split())
        .map(clean_spark_word)
        .filter(lambda word: word != "")
        .map(lambda word: (word, 1))
        .reduceByKey(lambda left, right: left + right)
        .collect()
)
```

Cette version est plus concise, car Spark cache les détails de communication. Elle permet cependant de comparer la solution MPI à une approche réellement utilisée dans les chaînes Big Data. Dans le contexte local et sur un petit fichier, Spark peut être désavantagé par son temps de démarrage et l'initialisation de la session JVM.

6. Interface web Streamlit

L'interface web rend le projet plus accessible pendant la soutenance. Elle permet de lancer MPI, lancer Spark, exécuter le benchmark complet, afficher les tables de Word Count et visualiser les résultats sous forme de graphiques.

Démonstration de l'interface Streamlit

Deploy

Mini MapReduce Engine with MPI

This project implements Map, Shuffle, and Reduce using MPI and compares it with Apache Spark.

Upload a .txt file

Upload

200MB per file • TXT

Using the default sample file from data/input.txt.

Number of MPI processes

4

Run MPI MapReduce

Run Apache Spark

Run Full Benchmark

Command finished successfully.

MPI MapReduce finished in 0.000716 seconds
Output written to /home/anass/Documents/Master-Ex/M1-S2/Systèmes/project/mini-mapreduce-mpi-web/results/mpi_output.txt

MPI Word Count

Spark Word Count

Deploy

MPI Word Count

Execution time

0.4748 s

word	count
is	4
mpi	4
and	3
phase	3
spark	3
the	3
a	2
apache	2
counts	2
implements	2

Spark Word Count

word	count
the	25
mpi	15
phase	15
data	10
engine	10
for	10
mapreduce	10
process	10
reduce	10
shuffle	10
spark	10

Deploy

Comparison

engine	processes	input_file	execution_time_seconds
MPI	2	/home/anass/Documents/Master-Ex/M1-S2/Systèmes/project/mini-mapreduce-mpi-web/data/sample_large	0.4864
MPI	4	/home/anass/Documents/Master-Ex/M1-S2/Systèmes/project/mini-mapreduce-mpi-web/data/sample_large	0.4721
MPI	6	/home/anass/Documents/Master-Ex/M1-S2/Systèmes/project/mini-mapreduce-mpi-web/data/sample_large	0.5147

Figure 3 - Captures de l'interface Streamlit : exécution, résultats Word Count et benchmark.

Fonction	Description
Upload du fichier texte	L'utilisateur peut importer un fichier .txt ou utiliser data/input.txt.
Choix du nombre de processus	Le nombre de processus MPI peut être fixé à 2, 4, 6 ou 8.
Boutons d'exécution	Trois actions : Run MPI MapReduce, Run Apache Spark, Run Full Benchmark.
Visualisation	Affichage des sorties, des tableaux de mots et des graphiques.
Explication pédagogique	Un bloc explique Map, Shuffle, Reduce et la différence MPI/Spark.

7. Protocole expérimental

7.1 Environnement et commandes utilisées

Les expériences sont exécutées sous Ubuntu dans un environnement virtuel Python. Les dépendances nécessaires sont Open MPI, mpi4py, PySpark, Streamlit, pandas et matplotlib.

```
sudo apt update
sudo apt install python3 python3-pip python3-venv openmpi-bin libopenmpi-dev openjdk-17-jdk -y
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

7.2 Commandes d'exécution

```
mpirun -np 4 python3 src/mini_mapreduce.py data/input.txt results/mpi_output.txt
python3 src/spark_wordcount.py data/input.txt results/spark_output.txt
python3 src/benchmark.py
streamlit run web/app.py
```

Exécutions terminal et benchmark

```
(venv) anass@anass-HP:~/Documents/Master-Ex/M1-S2/Systèmes/project/mini-
mapreduce-mpi-web$ mpirun -np 4 python3 src/mini_mapreduce.py data/input.txt results/mpi_output.txt
MPI MapReduce finished in 0.000580 seconds
Output written to results/mpi_output.txt
(venv) anass@anass-HP:~/Documents/Master-Ex/M1-S2/Systèmes/project/mini-

(venv) anass@anass-HP:~/Documents/Master-Ex/M1-S2/Systèmes/project/mini-
mapreduce-mpi-web$ python3 src/spark_wordcount.py data/input.txt results/spark_output.txt
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/06/16 17:57:14 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... us
ing builtin-java classes where applicable
[Stage 0:>                                     (0 +
Apache Spark finished in 9.896543 seconds
Output written to results/spark_output.txt
(venv) anass@anass-HP:~/Documents/Master-Ex/M1-S2/Systèmes/project/mini-

(venv) anass@anass-HP:~/Documents/Master-Ex/M1-S2/Systèmes/project/mini-
mapreduce-mpi-web$ python3 src/benchmark.py

Benchmark comparison
-----
Engine          Processes  Input file          Time (s)
-----
MPI              2          sample_large.txt    0.486357
MPI              4          sample_large.txt    0.472148
MPI              6          sample_large.txt    0.514740
Spark            local[*]    sample_large.txt    10.566103
-----
Saved to /home/anass/Documents/Master-Ex/M1-S2/Systèmes/project/mini-mapreduce-mpi-web/results/compar
ison.csv
(venv) anass@anass-HP:~/Documents/Master-Ex/M1-S2/Systèmes/project/mini-mapreduce-mpi-web$
```

Figure 4 - Captures terminal : exécution MPI, exécution Spark et benchmark.

7.3 Métriques observées

- le temps d'exécution en secondes ;
- la conformité des sorties entre MPI et Spark ;
- l'évolution du temps MPI lorsque le nombre de processus passe de 2 à 4 puis 6 ;
- l'impact du coût de démarrage de Spark en mode local.

8. Résultats expérimentaux et discussion

8.1 Résultats du benchmark

Moteur	Processus	Fichier	Temps (s)
MPI	2	sample_large.txt	0.486357
MPI	4	sample_large.txt	0.472148
MPI	6	sample_large.txt	0.514740
Spark	local[*]	sample_large.txt	10.566103

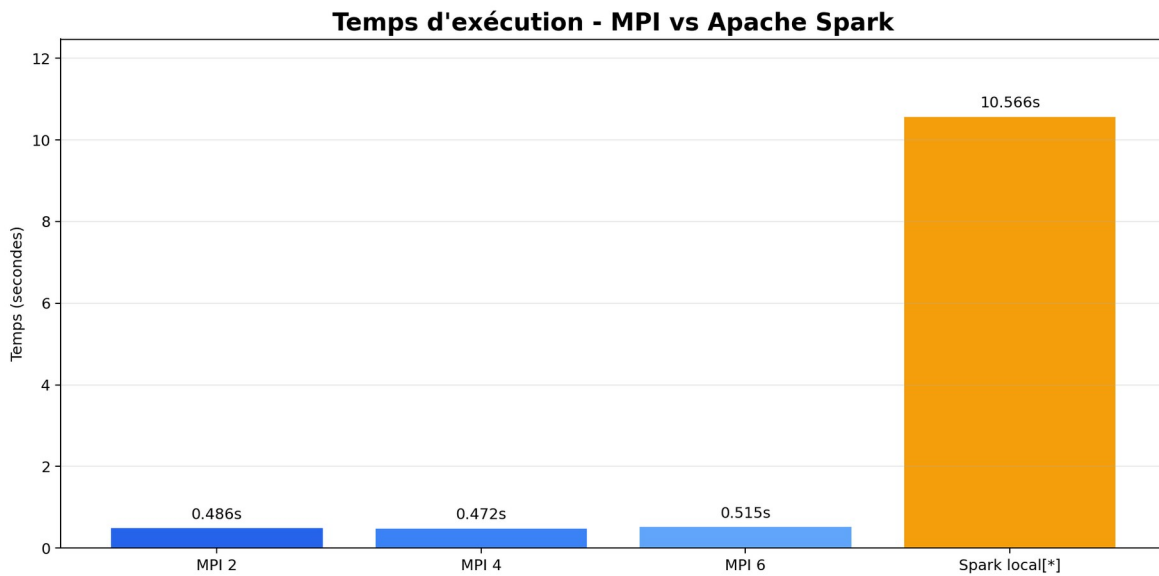


Figure 5 - Comparaison des temps d'exécution MPI et Spark local[*].

Le benchmark montre que la meilleure configuration observée est MPI avec 4 processus, avec un temps de 0,472148 seconde. Les configurations MPI à 2 et 6 processus restent proches, respectivement 0,486357 seconde et 0,514740 seconde. Spark local[*] prend 10,566103 secondes dans cette expérience.

8.2 Analyse de la scalabilité MPI

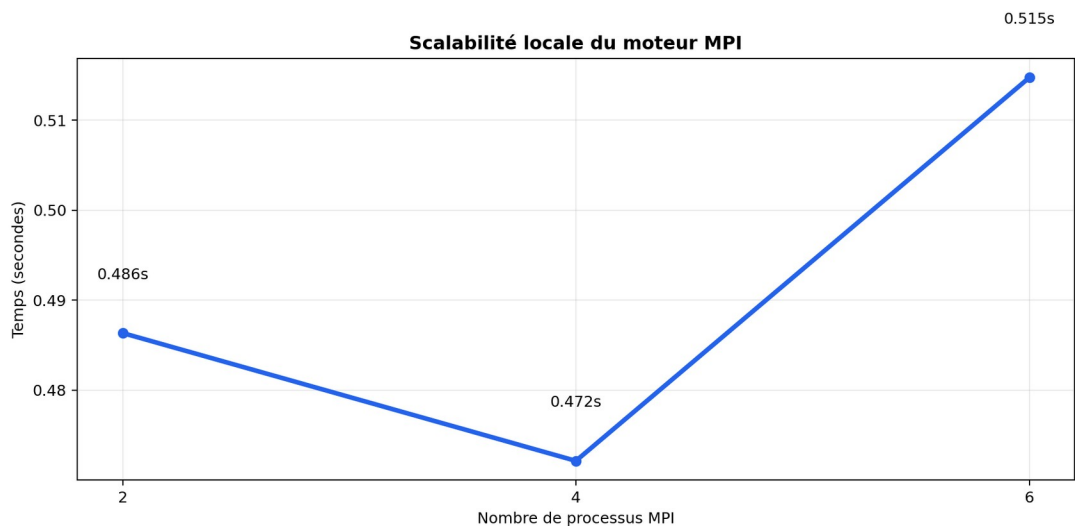


Figure 6 - Évolution du temps MPI en fonction du nombre de processus.

L'ajout de processus ne produit pas une accélération linéaire. Le passage de 2 à 4 processus donne une légère amélioration, mais le passage à 6 processus augmente le temps. Ce comportement est cohérent avec la petite taille du fichier de test : le coût de communication, de synchronisation et de lancement peut devenir plus important que le gain du parallélisme.

Configuration	Temps	Interprétation
MPI 2 processus	0.486357 s	Référence MPI locale.
MPI 4 processus	0.472148 s	Légère amélioration : speedup $\approx 1.03x$ par rapport à MPI 2.

MPI 6 processus	0.514740 s	Temps plus élevé : overhead supérieur au gain de parallélisation.
Spark local[*]	10.566103 s	Temps dominé par l'initialisation de Spark/JVM sur petit fichier.

8.3 Validation fonctionnelle des sorties

Pour le fichier `sample_large.txt`, les sorties `mpi_output_2.txt`, `mpi_output_4.txt`, `mpi_output_6.txt` et `spark_output.txt` sont identiques. Le même condensat SHA-256 est obtenu pour les quatre fichiers, ce qui valide la cohérence fonctionnelle entre l'implémentation MPI et la version Spark.

Fichier résultat	Empreinte SHA-256
<code>mpi_output_2.txt</code>	462783108e4be37c947188b9...
<code>mpi_output_4.txt</code>	462783108e4be37c947188b9...
<code>mpi_output_6.txt</code>	462783108e4be37c947188b9...
<code>spark_output.txt</code>	462783108e4be37c947188b9...

8.4 Analyse du Word Count

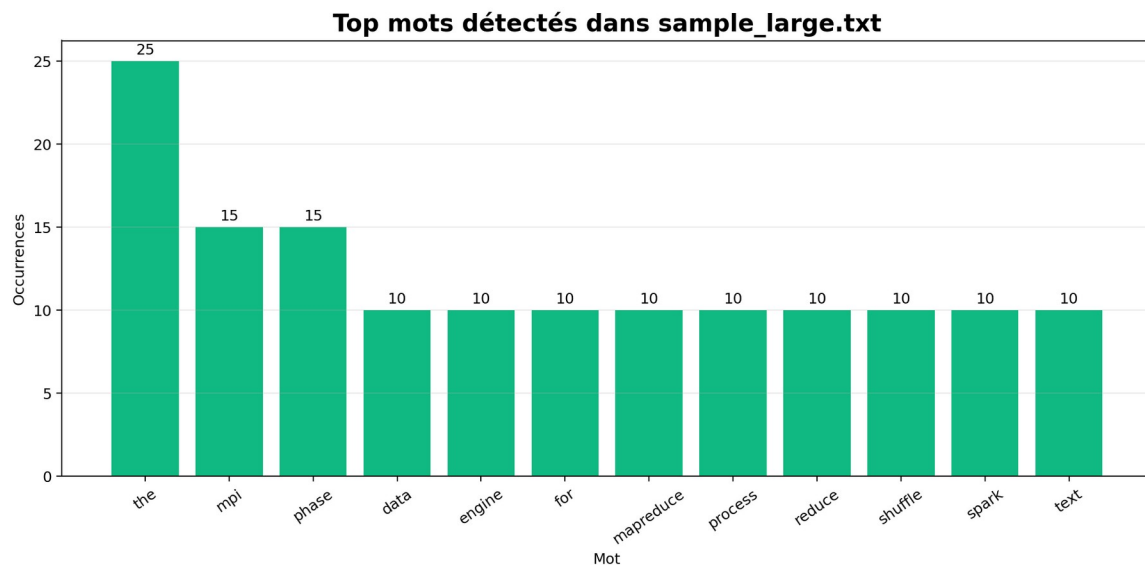


Figure 7 - Mots les plus fréquents détectés dans le fichier `sample_large.txt`.

Les mots les plus fréquents correspondent au vocabulaire volontairement répété dans le fichier de test : `the`, `mpi`, `phase`, `data`, `engine`, `mapreduce`, `reduce`, `shuffle` et `spark`. Cela confirme que le fichier de benchmark a été construit pour répéter les termes importants du projet et faciliter la vérification des résultats.

8.5 Interprétation de la comparaison MPI/Spark

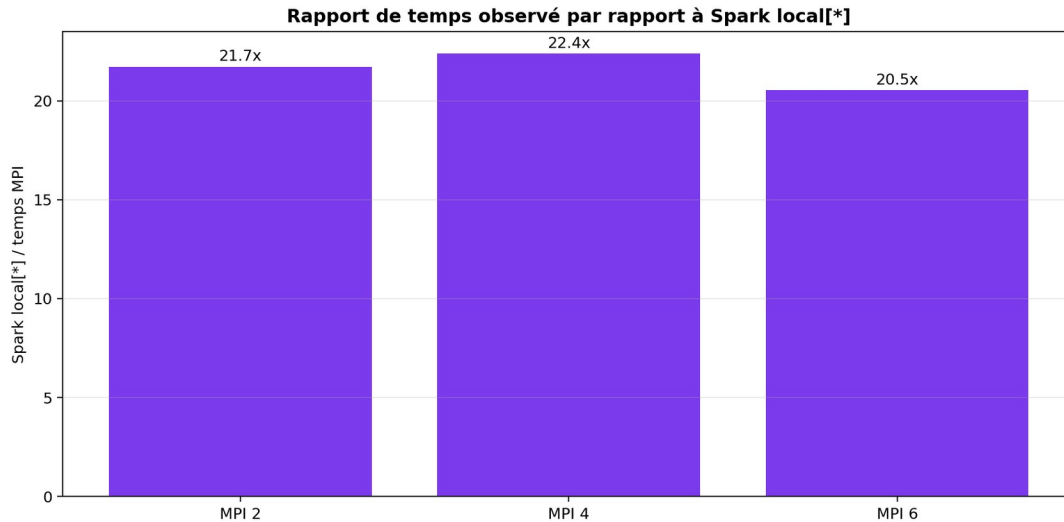


Figure 8 - Rapport de temps observé entre Spark local[*] et les configurations MPI.

Les résultats ne signifient pas que MPI est toujours plus rapide que Spark. Ils indiquent plutôt que, dans ce contexte local et sur un fichier de petite taille, Spark subit un coût de démarrage élevé. MPI est léger et démarre rapidement, ce qui explique son avantage dans l'expérience. Pour des volumes massifs, des traitements complexes ou un cluster réel, Spark devient plus pertinent grâce à son écosystème, sa planification automatique et sa tolérance aux pannes.

9. Comparaison synthétique MPI MapReduce / Apache Spark

Critère	MPI MapReduce	Apache Spark
Niveau d'abstraction	Bas niveau : communication explicite	Haut niveau : transformations de données
Contrôle	Très élevé : scatter, alltoall, gather écrits dans le code	Contrôle indirect : Spark gère l'exécution
Shuffle	Implémenté manuellement avec hash stable + alltoall	Géré automatiquement par Spark
Tolérance aux pannes	Non intégrée dans ce mini moteur	Intégrée dans l'écosystème Spark
Lisibilité pédagogique	Excellente pour comprendre les échanges	Bonne pour apprendre les pipelines Big Data
Scalabilité pratique	Possible mais demande une orchestration manuelle	Conçue pour clusters et gros volumes
Cas d'usage	Apprentissage, HPC, algorithmes parallèles	ETL, analytics, Big Data, production

10. Limites du projet et perspectives

10.1 Limites

- Les tests ont été réalisés localement et non sur un cluster multi-machines.
- Le fichier sample_large.txt reste petit ; les résultats ne représentent pas un scénario Big Data réel.
- Le mini moteur MPI ne fournit pas de tolérance aux pannes automatique.

- Le benchmark utilise une mesure simple ; des moyennes sur plusieurs répétitions seraient plus robustes.
- Le projet traite uniquement le Word Count, même si le modèle pourrait être généralisé.

10.2 Perspectives d'amélioration

- exécuter MPI sur plusieurs machines physiques ou virtuelles ;
- tester des fichiers de plusieurs centaines de mégaoctets ou plus ;
- ajouter d'autres tâches MapReduce comme l'index inversé ou le tri distribué ;
- intégrer un stockage distribué tel que HDFS ;
- ajouter des répétitions automatiques au benchmark et calculer moyenne, médiane et écart-type ;
- améliorer l'interface Streamlit avec historique des exécutions et export PDF/Excel.

11. Conclusion

Ce projet a permis de développer un mini moteur MapReduce avec MPI et de comprendre concrètement les mécanismes internes d'un traitement distribué. Contrairement à une solution de haut niveau, l'implémentation MPI oblige à gérer explicitement la distribution des données, la redistribution des clés et l'agrégation des résultats. Cette approche est très formatrice pour comprendre ce qui se passe derrière les frameworks Big Data.

La comparaison avec Apache Spark montre deux visions complémentaires. MPI offre un contrôle fin et une faible surcharge pour des expérimentations simples. Spark propose une abstraction plus productive, mieux adaptée à des applications Big Data réelles et à des environnements distribués complexes. L'interface Streamlit complète le projet en rendant les exécutions et les résultats faciles à présenter.

En conclusion, le projet atteint ses objectifs : implémenter Map, Shuffle et Reduce, valider les résultats par comparaison avec Spark, mesurer les performances et proposer une démonstration interactive. Il constitue une base solide pour approfondir le calcul parallèle, les systèmes distribués et les frameworks de traitement de données.

12. Bibliographie et ressources

Référence	Description
Dean, J. et Ghemawat, S.	MapReduce: Simplified Data Processing on Large Clusters, OSDI, 2004.
Open MPI	Documentation officielle du projet Open MPI.
mpi4py	Documentation officielle de mpi4py pour l'utilisation de MPI en Python.
Apache Spark	Documentation officielle Apache Spark et PySpark.
Streamlit	Documentation officielle Streamlit pour la création rapide d'interfaces web Python.

Annexe A - Commandes principales

```
# MPI avec 2, 4 et 6 processus
mpirun --oversubscribe -np 2 python3 src/mini_mapreduce.py data/sample_large.txt
results/mpi_output_2.txt

mpirun --oversubscribe -np 4 python3 src/mini_mapreduce.py data/sample_large.txt
results/mpi_output_4.txt

mpirun --oversubscribe -np 6 python3 src/mini_mapreduce.py data/sample_large.txt
results/mpi_output_6.txt

# Spark
python3 src/spark_wordcount.py data/sample_large.txt results/spark_output.txt

# Benchmark
python3 src/benchmark.py

# Interface web
streamlit run web/app.py
```

Annexe B - Exemple de sortie Word Count

```
the 25
mpi 15
phase 15
data 10
engine 10
for 10
mapreduce 10
process 10
reduce 10
shuffle 10
spark 10
text 10
word 10
a 5
aggregates 5
and 5
apache 5
benchmarking 5
between 5
communication 5
computing 5
control 5
count 5
counts 5
demonstrates 5
direct 5
distributed 5
final 5
gives 5
groups 5
```


Annexe D - Extrait du benchmark CSV

```
engine,processes,input_file,execution_time_seconds
MPI,2,/home/anass/Documents/Master-Ex/M1-S2/Systèmes/project/mini-mapreduce-mpi-web/data/
sample_large.txt,0.486357
MPI,4,/home/anass/Documents/Master-Ex/M1-S2/Systèmes/project/mini-mapreduce-mpi-web/data/
sample_large.txt,0.472148
MPI,6,/home/anass/Documents/Master-Ex/M1-S2/Systèmes/project/mini-mapreduce-mpi-web/data/
sample_large.txt,0.514740
Spark,local[*],/home/anass/Documents/Master-Ex/M1-S2/Systèmes/project/mini-mapreduce-mpi-web/
data/sample_large.txt,10.566103
```